Data Validation Tool

for Machine Learning Datasets

Antonis Paterakis

github.com/antonispaterakis/data-validator-mcp

The Problem

✗

Bad labels = bad models

A model trained on mislabeled data inherits those errors — and nobody checks systematically before training.

✗

Manual review doesn't scale

Checking 10,000 rows by hand is impractical. Most teams skip it and discover problems after deployment.

✗

Brute-force LLM scan is expensive

Sending every row to an LLM for validation costs too much — both in time and API tokens.

The Idea

Use geometry to filter, then LLM to judge.

Instead of scanning every row with an LLM (slow, expensive), we first embed the text into vector space. Rows that are semantically similar end up close to each other. If a row's label disagrees with its neighbours, it becomes suspicious — and only those go to the LLM for judgment.

The cluster-first, LLM-second principle is inspired by the paper "Large Language Models for Efficient Topic Modeling" (Theocharopoulos et al., Neural Computing and Applications).

What is MCP?

terminology

Model Context Protocol

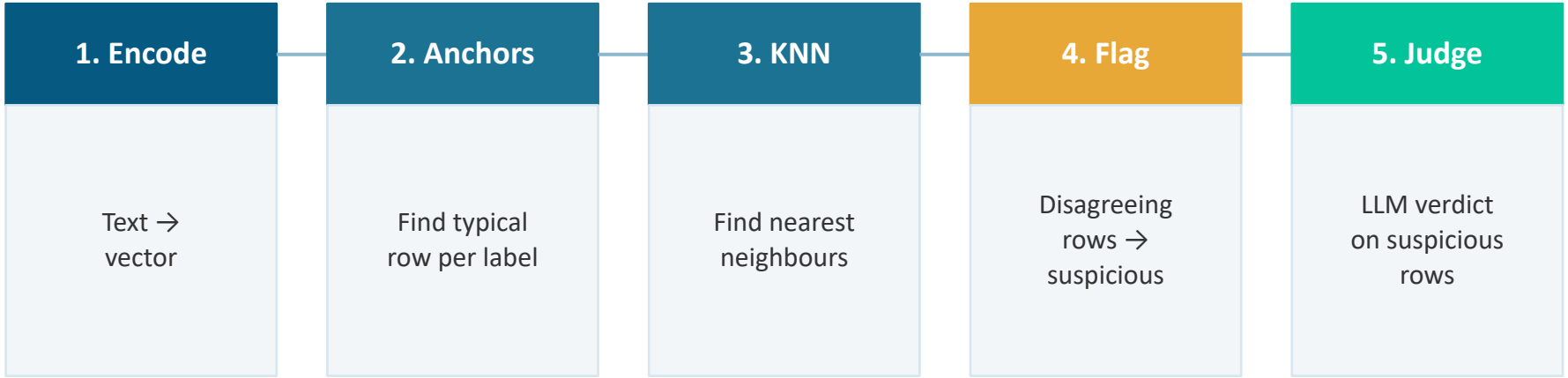
MCP is an open standard released by Anthropic in late 2024. It defines how an LLM like Claude can call external tools — read files, query APIs, run code — in a consistent way. An MCP server exposes a set of tools; the LLM decides when to call them based on the user's conversation.

Why we use it

→ Makes the validator usable from Claude Desktop directly
→ Instead of running a Python script, the user says "validate this CSV" and Claude calls our tool
→ Standardized: anyone with Claude Desktop + the server installed can use it
→ Recommended by Prof. Plagianakos as a foundational tool to learn

The Pipeline

Five stages, from CSV input to flagged rows output.



Each stage is explained individually in the next slides.

Step 1 — What are embeddings?

terminology

An embedding is a list of numbers that represents the meaning of a piece of text. Texts that mean similar things produce similar numbers, so you can compare them mathematically.

"chest pain and shortness of breath" → `[0.12, -0.44, 0.87, ..., 0.31]`

(384 numbers)

"hepatic failure from sepsis" → `[0.81, 0.23, -0.15, ..., -0.62]`

Why we use them:

We cannot mathematically compare raw text, but we can compare vectors. Once every row is an embedding, we can measure how similar two rows are — which is the whole foundation of finding neighbours in step 3.

Step 1 tool — all-MiniLM-L6-v2

terminology

The specific embedding model we use. It's open-source and runs locally.

Decoding the name:

all — trained on general-purpose data, not a specific domain
MiniLM — a smaller, distilled version of BERT (Google's 2018 model that changed NLP)
L6 — 6 transformer layers (full BERT has 12; fewer = faster, still accurate)
v2 — second revision, better training data than v1

Why this model:

→ Small (90 MB), runs on CPU, no GPU needed
→ Trained specifically for sentence similarity (output 384-dim vectors)
→ Industry-standard baseline for embedding tasks — documented benchmarks
→ Loaded via the sentence-transformers Python library

Step 2 — Select anchors

For each label, find the single most representative row.

For every label, we compute the average vector of all rows with that label (the centroid), then pick the row whose embedding is closest to this centroid. That row becomes the "anchor" — a known-good example of what the label looks like in practice.

Why this is important:

When the LLM later judges a suspicious row, we hand it the anchor as a reference.
Instead of asking "is this row cardiology?" in a vacuum, we ask
"here's a confirmed cardiology example — does this row belong with it?"
This gives the LLM a concrete comparison point and reduces hallucinated verdicts.

Step 3 — Find nearest neighbours

terminology

KNN = K-Nearest Neighbours. Cosine similarity = how we measure closeness.

KNN

For each row, find the K=5 rows whose embeddings are most similar. These 5 are its "neighbours" in vector space.

Why K=5:

Small enough to be fast, large enough to vote reliably. We tested K=15 — too slow on long texts.

Cosine similarity

A number between -1 and 1 measuring the angle between two vectors. 1 = identical direction, 0 = unrelated, -1 = opposite.

Why cosine (not distance):

For text embeddings, direction carries meaning more than magnitude. It's the industry-standard metric.

Step 4 — Flag suspicious rows

If a row's label disagrees with most of its neighbours, it's suspicious.

We compute agreement = (neighbours sharing the row's label) / K. Rows where agreement is below 0.5 are flagged — meaning most neighbours have a different label.

Example: agreement = 0.8 ✓ keep

Row labelled: cardiology

Neighbours' labels:

cardiology, cardiology, cardiology,

cardiology, neurology

→ 4/5 match = looks correct

Example: agreement = 0.2 ✗ flag

Row labelled: cardiology

Neighbours' labels:

digestive, digestive, digestive,

digestive, cardiology

→ 1/5 match = suspicious, send to LLM

Step 5 — LLM Judge

terminology

A Large Language Model makes the final verdict on suspicious rows.

Prompt context per flagged row:

→ The row's text

→ The anchor for that label (from step 2)

→ Its assigned label

→ The 5 nearest neighbours + their labels

Why a local LLM (not a cloud API):

→ Zero API cost — we process hundreds of rows for free
→ No data leaves the machine (important for medical/private datasets)
→ Served via LM Studio / Ollama (both expose an OpenAI-compatible endpoint)
→ Model used: Llama 3.2 — small, fast, good at structured JSON output

Results — Sanity check

Small synthetic test to verify the pipeline works end-to-end.

Setup:

→ 12 rows, 2 labels: cardiology and orthopedics
→ 2 rows intentionally mislabeled (row 10 and row 11 — labels swapped)
→ Expected result: the pipeline should find exactly these 2 rows

2 / 2	6×	748	0
Mislabels found	Efficiency vs brute force	Tokens used	False positives

Results — Sanity check analysis

What we observed

Row 10 (labelled orthopedics) and row 11 (labelled cardiology) were both flagged as suspicious and judged as bad by the LLM. The LLM also suggested the correct labels (swapping them back). No other rows were flagged.

Result

Pipeline works: 100% recall on known mislabels, 0% false positives, and 6x fewer tokens than brute-force scanning every row.

Why it worked

Cardiology and orthopedics are semantically very different. Their embeddings live in separate regions of vector space, so neighbours are almost always the same label — making any disagreement stand out clearly.

Results — Real dataset

Test on a public medical dataset to see how the pipeline handles real data.

Setup:

→ Dataset: 123rc/medical_text from HuggingFace (open, Apache 2.0 license)
→ 200 medical abstracts sampled from the training split
→ 5 labels: neoplasms, digestive_diseases, nervous_system_diseases, cardiovascular_diseases, general_pathological
→ Unknown true mislabel count — this is a discovery task, not validation

153 / 200	153	0	1.2×
Rows flagged by KNN	LLM confirmed bad	LLM good verdicts	Efficiency ratio

Real dataset — what happened

What we observed

The pipeline flagged 153 out of 200 rows as suspicious — implying a 76% mislabel rate, which is implausible. The LLM then confirmed every single flag as bad, with zero good verdicts.

Result

The pipeline collapsed on this dataset. 1.2× efficiency means we barely saved any tokens vs. brute force, and the signal is unreliable. Not useful output.

Initial hypothesis

Two things failed together: the KNN agreement threshold was too strict, and the LLM seemed to rubber-stamp every flag. But these are symptoms, not the cause. Next slide digs into the root cause.

Real dataset — root cause

The embedding geometry does not support this label taxonomy.

When cardiology vs orthopedics (sanity check) were used, the two categories used very different vocabulary and lived in distinct regions of vector space. The 5 medical abstract categories do not.

The geometric problem

All 5 categories share medical jargon, statistical language, and clinical framing.
A paper on hepatic failure from sepsis looks almost identical to one on cardiac failure in the embedding space. The categories overlap — they are not geometrically separable.

Cascade effect: overlapping embeddings → neighbours disagree almost always → everything flagged → LLM receives weak context → rubber-stamps every verdict

How to run the tool

Where the project lives

Local path: **~/data-validator-mcp/** *(my laptop)*
GitHub: github.com/antonispaterakis/data-validator-mcp

Key files

server.py	MCP server entry point, exposes the tools to Claude
pipeline.py	Orchestrates encode → anchors → KNN → flag → judge
embeddings.py	Wraps all-MiniLM-L6-v2 + anchor selection
llm_judge.py	Calls LM Studio / Ollama, tracks tokens
test_dataset.csv	The 200-row medical sample we tested on

Running it

1. Start LM Studio (or Ollama), load Llama 3.2, run the local server
2. Open Claude Desktop — the MCP server is registered in claude_desktop_config.json
3. Prompt Claude: "Validate ~/data-validator-mcp/test_dataset.csv with text and label columns"